AROGYAMITRA HEALTHCARE PLATFORM

Comprehensive Production-Readiness Security Audit Report

Target: Backend (Express/Node.js, PostgreSQL, S3) | WebFrontend (React 19) | Application (React Native) | CI/CD Workflows
Audit Date: September 2026 | Standard: OWASP Top 10 (2021) & OWASP API Top 10 (2023) | Scope: Static Code & Config Audit

SECURITY READINESS SCORE

23%

VERDICT: NOT READY

Critical security hardening required before handling patient data.

AUDIT CONTROL METRICS

Total Security Categories Evaluated	**44 Controls**
Fully Implemented Controls (100%)	**1 (2.3%)**
Partially Implemented Controls (50%)	**18 (40.9%)**
Missing / Not Implemented Controls (0%)	**8 (18.2%)**
Risky / Incorrect Implementations (0%)	**17 (38.6%)**
Unverifiable / Excluded Controls	**1 (NoSQL N/A)**

EXECUTIVE SECURITY POSTURE SUMMARY

A rigorous, zero-assumption source-code security audit of the entire ArogyaMitra repository reveals critical vulnerabilities that prevent production deployment in its current state. While core software engineering practices are present—such as parameterized PostgreSQL queries, scrypt password hashing, and clean MVC layered architecture—the application exhibits severe authorization and authentication defects:

1. OTP Bypass in All Logins: The API returns plain-text OTPs in JSON HTTP responses for Citizen, Sevak, and Admin logins, allowing instant account takeover without an SMS gateway.
2. Public Privilege Escalation: POST /api/users is unauthenticated and accepts arbitrary role parameters, allowing anyone to register as an Administrator.

CRITICAL SECURITY VULNERABILITIES REQUIRING IMMEDIATE REMEDIATION

unauthenticated or optionalJWT-guarded routes.

1. Complete Authentication Bypass & Account Takeover (Leaked OTP)

Location: Backend/src/services/auth.service.js, citizen.service.js, arogyaSevak.service.js

Plaintext OTP returned in HTTP JSON response. Any account (Admin, Sevak, Citizen) can be hijacked immediately.

2. Unauthenticated Admin Creation & Privilege Escalation

Location: Backend/src/routes/user.routes.js, Backend/src/validations/user.validation.js

POST /api/users is public and accepts role: "ADMIN". Anyone can create a super administrator account.

3. Mass Broken Object-Level Authorization (BOLA/IDOR) & Patient Data Exposure

Location: Backend/src/routes/patient.routes.js, application.routes.js, opdVisit.controller.js

Patient records, diagnoses, and medical records are returned to unauthenticated requests via optionalJWT.

4. Public Notification Broadcast & Phishing Injection

Location: Backend/src/routes/mobile.routes.js, Backend/src/controllers/notification.controller.js

POST /api/mob/notifications/broadcast lacks authentication; arbitrary callers can broadcast system-wide alerts.

5. Man-In-The-Middle (MITM) on Cloud Database via Disabled TLS Validation

Location: Backend/src/config/db.js (ssl: { rejectUnauthorized: false })

Disables SSL certificate verification for remote PostgreSQL. Intercepting proxies can harvest credentials and PHI.

AROGYAMITRA HEALTHCARE PLATFORM

Production Security Framework - 44 Evaluated Security Controls

#	Security Area	Status	Score	Key Evidence & Implementation Finding	Risk	Required Fix
1	**HTTPS / TLS**	RISK	0%	server.js binds HTTP; Android usesCleartextTraffic=true	CRIT	Terminate TLS on ALB; enforce strict HTTPS
2	**Authentication**	RISK	0%	auth.service.js & citizen.service.js return OTP in API JSON	CRIT	Strip OTP from response; integrate carrier SMS
3	**Password Security**	PARTIAL	50%	scrypt used; weak policy (len>=6); seedDefaultAdmin hardcoded	HIGH	Enforce 12+ chars, upper/lower/num/sym; remove seed pwd
4	**JWT Security**	PARTIAL	50%	No HS256 algorithm pinning; 1d long lifespan; fallback secrets	HIGH	Pin algorithms: ["HS256"]; 15m expiry; enforce secrets
5	**Refresh Tokens**	PARTIAL	50%	Stored in DB; unhashed; no token rotation (RTR) on renew	HIGH	Implement refresh token rotation; hash in DB
6	**Session Security**	RISK	0%	Tokens saved in browser localStorage and React Native AsyncStorage	HIGH	Migrate to HttpOnly, Secure, SameSite=Strict cookies
7	**Authorization / RBAC**	RISK	0%	POST /api/users is public and accepts role: "ADMIN"	CRIT	Lock user creation to Admin; forbid role in public signups
8	**Object-Level Auth (BOLA)**	RISK	0%	optionalJWT on /api/patients; anyone reads any patient PHI	CRIT	Enforce strict record ownership checks in controllers
9	**Input Validation**	PARTIAL	50%	Zod schemas present on select routes; missing on patient updates	MED	Enforce Zod schemas across 100% of mutation routes
10	**SQL Injection Defense**	PASS	100%	Parameterized queries via pg.Pool with $1, $2 placeholders	LOW	Maintain parameterization standards across all queries
11	**XSS Protection**	RISK	0%	Reports.jsx document.write unescaped HTML; SVG upload enabled	HIGH	Sanitize via DOMPurify; forbid SVG or sanitize XML
12	**CSRF Protection**	MISSING	0%	No CSRF tokens; credentials:true with wildcard origins	MED	Add csrf-csrf middleware for browser-facing sessions
13	**CORS Configuration**	RISK	0%	Regex allows any *.vercel.app domain with credentials:true	HIGH	Whitelist exact production FQDNs only
14	**Security Headers**	PARTIAL	50%	helmet() on backend; no CSP or HSTS headers on WebFrontend	MED	Configure strict CSP and HSTS on CloudFront/Vercel
15	**Rate Limiting**	PARTIAL	50%	express-rate-limit in-memory; only active if NODE_ENV=prod	MED	Move counters to Redis; attach to dev and prod
16	**Brute-Force Protection**	PARTIAL	50%	loginRateLimiter tracks mobile; missing on /send-otp routes	HIGH	Apply rate limiting to all OTP generation endpoints
17	**Account Lockout**	PARTIAL	50%	15-min in-memory lockout; resets on PM2 restart/cluster	MED	Store lockout state in Redis or database table
18	**API Security Architecture**	RISK	0%	Unauthenticated broadcast alerts; 50MB JSON parser body limit	CRIT	Lock administrative routes; reduce JSON limit to 2MB
19	**File-Upload Security**	PARTIAL	50%	Presigned S3 keys generated server-side; optionalJWT allows bypass	HIGH	Require verified JWT before presigned URL generation
20	**Malware / File Validation**	RISK	0%	Checks MIME via client header; no magic byte signature check	HIGH	Implement file-type buffer check; add GuardDuty scanning
21	**Database Security**	RISK	0%	ssl: { rejectUnauthorized: false } allows MITM on remote DB	CRIT	Enforce rejectUnauthorized:true with CA root bundle
22	**Database Least Privilege**	MISSING	0%	Connects as superuser/owner; runs DDL migrations on startup	HIGH	Split roles: DDL migration role vs DML runtime user
23	**Secrets Management**	MISSING	0%	Plain .env on EC2; default fallback secrets in source code	HIGH	Use AWS Secrets Manager or Systems Manager Parameter Store
24	**Env Variable Security**	PARTIAL	50%	Object.freeze() in config; silent dangerous fallback defaults	MED	Fail startup immediately if critical secrets are unset
25	**AWS IAM Security**	PARTIAL	50%	OIDC used in frontend CI; backend uses static access keys	MED	Attach IAM Instance Profile to EC2; remove static keys
26	**S3 Storage Security**	RISK	0%	Direct public S3 URLs constructed for confidential medical PHI	CRIT	Block All Public Access; stream via presigned GET URLs
27	**Redis Security**	MISSING	0%	No Redis deployed; application uses single-process volatile Maps	MED	Deploy AWS ElastiCache Redis with TLS and AUTH
28	**Encryption at Rest**	PARTIAL	50%	Relies on cloud disk encryption; no field-level encryption for PHI	HIGH	Apply AES-256-GCM field encryption for Aadhaar & diagnosis
29	**Encryption in Transit**	RISK	0%	Database TLS unverified; mobile allows HTTP; EC2 port 5000 HTTP	CRIT	Enforce TLS 1.3 across all network links and clients
30	**Application Logging**	RISK	0%	console.log outputs plaintext OTPs and phone numbers to logs	HIGH	Sanitize loggers via Winston/Pino; mask sensitive PII
31	**Audit Logging**	PARTIAL	50%	AuditLogModel has PII sanitizer; only used by archival job	HIGH	Log all authentication, role changes, and PHI access
32	**Error Handling**	PARTIAL	50%	ApiError handler; leaks raw DB messages and dev stack traces	MED	Mask internal database error messages; suppress stack traces
33	**Dependency Security**	PARTIAL	50%	Vulnerabilities in qs, body-parser, and mobile image-size	HIGH	Update npm dependencies to remediate active CVEs
34	**npm / Package Security**	PARTIAL	50%	package-lock.json used; no automated vulnerability gate in CI	LOW	Add npm audit --audit-level=high step in GitHub Actions
35	**Git Secret Protection**	RISK	0%	.env files were previously committed in git history	HIGH	Scrub git history with git-filter-repo; rotate secrets
36	**Docker / Containers**	MISSING	0%	No Dockerfiles; deployed directly via PM2 on host OS	LOW	Build non-root, minimal distroless container images
37	**CI/CD Pipeline Security**	PARTIAL	50%	OIDC debugging dumps tokens; SSH pull without code signing	MED	Remove token claim dump; enforce signed commit builds
38	**Server / EC2 Security**	RISK	0%	Node runs directly on host; no reverse proxy (Nginx) configured	HIGH	Deploy hardened Nginx reverse proxy; place EC2 in private VPC
39	**AWS WAF / DDoS Defense**	MISSING	0%	No WAF rules, Shield, or CloudFront rate limiting configured	HIGH	Deploy AWS WAF with Core Rule Set ahead of the API
40	**Backup & Recovery**	MISSING	0%	No automated database backup scripts or disaster recovery plan	HIGH	Configure PostgreSQL automated daily snapshots and PITR
41	**Monitoring & Alerting**	MISSING	0%	Only basic health check; no APM (Sentry/CloudWatch Alarms)	MED	Integrate Sentry for error tracking and CloudWatch metrics
42	**Security Testing**	PARTIAL	50%	Regression scripts present; no automated SAST/DAST in CI	MED	Integrate Semgrep SAST and TruffleHog secret scanning
43	**OWASP Top 10 (2021)**	RISK	0%	Active failures across A01, A02, A03, A04, A05, A07, A09	CRIT	Execute remediation roadmap phases 1 through 4
44	**OWASP API Top 10 (2023)**	RISK	0%	Active failures across API1, API2, API3, API4, API5	CRIT	Implement object authorization, rate limiting, and RBAC

AROGYAMITRA HEALTHCARE PLATFORM

Critical Vulnerability Deep Dive & Conceptual Exploit Analysis

ISSUE 1: Complete Authentication Bypass & Account Takeover (Leaked OTP)

Severity: CRITICAL (CVSS 9.8) - CWE-287 / CWE-312

Location: Backend/src/services/auth.service.js (lines 107-108, 186); citizen.service.js (line 313); arogyaSevak.service.js (line 144)

Root Cause & Mechanics:

The application implements OTP verification for login, first-time password setup, and password resets. However, the server includes the generated OTP directly in the JSON response body under data.otp. Furthermore, generated OTPs are logged in plaintext to stdout using console.log and logger.info.

Conceptual Attack Scenario:

1. Attacker calls POST /api/v1/auth/send-otp with the mobile number of System Admin (9999999999).
2. API returns HTTP 200 with {"message": "OTP sent successfully", "otp": "5831"}.
3. Attacker submits the leaked OTP to /api/v1/auth/verify-otp, obtaining a valid session and bypassing 2FA.
4. Attacker calls /set-password or /register-password to overwrite the password and permanently seize control.

Required Remediation:

Immediately remove the otp field from all controller return objects. Integrate an official carrier SMS gateway (AWS SNS / Twilio). Increase OTP length to 6 digits, hash stored OTPs, and purge all logging statements.

ISSUE 2: Unrestricted Public Privilege Escalation to System Administrator

Severity: CRITICAL (CVSS 9.8) - CWE-269 / CWE-284

Location: Backend/src/routes/user.routes.js (line 11); Backend/src/validations/user.validation.js (lines 54-59)

Root Cause & Mechanics:

The route POST /api/users is mounted publicly without verifyJWT middleware. The input validation schema accepts a client-provided role parameter (ADMIN, CITIZEN, SEVAK) and directly writes it into public.users.

Conceptual Attack Scenario:

1. An unauthenticated adversary sends a POST request to /api/v1/users.
2. Body payload contains {"full_name": "Rogue Admin", "mobile": "9876500000", "password": "SecretPassword123!", "role": "ADMIN"}.
3. The backend executes an INSERT into public.users with role = "ADMIN".
4. Adversary logs into the Web Admin Portal, receiving full administrative authority over all hospital and patient records.

Required Remediation:

Attach verifyJWT and authorizeRoles("ADMIN") to user.routes.js. Enforce role assignment exclusively via private administrative channels; never trust client-supplied role values during public onboarding.

ISSUE 3: Mass Protected Health Information (PHI) Exposure (BOLA / IDOR)

Severity: CRITICAL (CVSS 9.3) - CWE-639 / CWE-200

Location: Backend/src/routes/patient.routes.js (lines 12-25); Backend/src/controllers/patient.controller.js (lines 26-31)

Root Cause & Mechanics:

The route GET /api/patients/assigned is guarded only with optionalJWT. When called without an Authorization header, req.user is null, triggering a fallback branch that executes ApplicationModel.findAll({ limit: 50 }). Complete patient medical records are returned without verifying the caller.

Conceptual Attack Scenario:

1. Any unauthenticated crawler makes a GET request to /api/v1/patients/assigned.
2. Server responds with 50 patient records, exposing patient names, ages, phone numbers, attending hospitals, diagnoses, and medical condition notes.
3. Similarly, /api/v1/applications/:id/documents allows unauthenticated retrieval of uploaded Aadhaar cards and clinical scans.

Required Remediation:

Enforce mandatory verifyJWT authentication across all /api/patients and /api/applications routes. Implement row-level access control: Citizens can only query records where citizen_user_id === req.user.id.

ISSUE 4: Unauthenticated Public Broadcast of System-Wide Alerts

Severity: CRITICAL (CVSS 8.6) - CWE-306 / CWE-285

Location: Backend/src/routes/mobile.routes.js (line 56); Backend/src/controllers/notification.controller.js (lines 42-62)

Root Cause & Mechanics:

The mobile notification broadcast endpoint POST /api/mob/notifications/broadcast is guarded only by optionalAuth. If unauthenticated, req.user?.id resolves to undefined, but execution continues and writes an active broadcast notice to public.notifications.

Conceptual Attack Scenario:

1. Remote attacker submits POST /api/mob/notifications/broadcast with title "Emergency Alert" and message containing a malicious phishing URL.
2. The system publishes the notice to all citizens and Sevaks across web and mobile clients.

Required Remediation:

Require verifyJWT and authorizeRoles("ADMIN") on all notification dispatch routes.

AROGYAMITRA HEALTHCARE PLATFORM

High & Medium Risks / Common Security Anti-Patterns Checklist

& þ HIGH & MEDIUM SECURITY RISKS REQUIRING ATTENTION

Insecure CORS Permitting Arbitrary Vercel Deployments (HIGH)

File: Backend/src/app.js (line 62)

CORS origin check uses regex /^https:\/\/([a-zA-Z0-9_-]+\.)*vercel\.app$/ with credentials:true. Any attacker hosting an app on Vercel can make credentialed cross-origin calls to steal user sessions.

Fix: Remove regex; whitelist exact approved production origins.

Stored DOM Cross-Site Scripting (XSS) in Report Printing (HIGH)

File: WebFrontend/src/pages/admin/Reports/Reports.jsx (lines 747-759)

Hospital names (${h.name}) are interpolated into raw HTML strings and written to a popup window via document.write(). Allows malicious hospital names to execute arbitrary script.

Fix: Sanitize strings using DOMPurify before document.write; set Content Security Policy.

Cleartext Transmission of PHI on Android Mobile App (HIGH)

File: Application/android/app/src/main/AndroidManifest.xml (line 17)

android:usesCleartextTraffic="true" is configured. Mobile app transmits medical records and tokens over unencrypted HTTP, allowing interception on local hospital Wi-Fi networks.

Fix: Set usesCleartextTraffic="false"; configure network_security_config.xml.

Direct Public URLs for Sensitive S3 Documents (HIGH)

File: Backend/src/services/s3.service.js (lines 34, 118)

Constructs public S3 URLs for identity documents, income certificates, and clinical records. Exposes records publicly if bucket permissions are misconfigured.

Fix: Block All Public Access; stream documents exclusively via time-limited presigned GET URLs.

Sensitive Plaintext Credentials & OTPs Stored in Server Logs (HIGH)

File: Backend/src/services/auth.service.js (lines 98, 176); citizen.service.js

console.log and logger.info print active OTPs and phone numbers to server logs on every login request. Anyone with log access can hijack user sessions.

Fix: Remove all credential logging; implement automated PII masking in Winston/Pino.

In-Memory State Breaking PM2 Cluster Mode & Scalability (MEDIUM)

File: Backend/src/middlewares/rateLimiter.middleware.js; Backend/src/services/auth.service.js

Rate limiting and OTP storage use in-memory Maps. Multi-core PM2 instances cannot share state, and server restarts completely wipe rate limit quotas and lockout states.

Fix: Deploy Redis (AWS ElastiCache) with rate-limit-redis to centralize session state.

50MB Request Body Limit Enabling Denial-of-Service (MEDIUM)

File: Backend/src/app.js (lines 98-99)

Express JSON parser allows 50MB bodies. Attackers can flood the API with large JSON objects, exhausting V8 heap memory and starving the Event Loop.

Fix: Lower body limit to 2MB; handle document uploads via direct presigned S3 PUTs.

Ø=Ý COMMON SECURITY ANTI-PATTERNS CHECKLIST

Hardcoded Passwords seedDefaultAdmin.js contains plaintext admin password	YES	**Hardcoded JWT Secret** environment.js has default fallback secret	YES
Hardcoded DB Credentials db.js falls back to postgres:postgres	YES	**Committed .env Files** Git history shows committed Application/.env and WebFrontend/.env	YES
CORS Wildcard (*) app.js defaults to wildcard * if env var unset	YES	**Raw SQL Injection** pg.Pool parameterized queries ($1, $2) used systematically	NO
Weak Password Hashing crypto.scrypt with 16-byte random salt used	NO	**JWT Expiration > 1 Hour** Default access token expiration is 1 day (1d)	YES
Refresh Tokens Rotated Same refresh token reused across sessions	NO	**IDOR / BOLA Checks** Missing on /api/patients and /api/applications/:id	FAIL
Credentials in Logs Plaintext OTPs logged via console.log and logger.info	FAIL	**Unrestricted Uploads** SVG allowed; MIME check relies on client headers	FAIL

AROGYAMITRA HEALTHCARE PLATFORM

Production Architecture Review & 5-Phase Hardening Roadmap

Ø<ß×þ ARCHITECTURE PRODUCTION-READINESS REVIEW

Target Production Architecture Topology:

Frontend (React / Mobile App) --> CDN (CloudFront) --> AWS WAF --> Application Load Balancer (TLS 1.3) --> Node.js API (EC2 Private VPC) --> Redis Cluster --> PostgreSQL (Supabase/RDS) & S3 (Private)

Identified Missing Critical Architectural Layers:

1. Missing AWS WAF: No web application firewall inspecting traffic for SQLi, XSS, and rate flooding.
2. Missing In-Transit TLS on Server: Node runs cleartext HTTP on port 5000 without an ALB/Nginx reverse proxy.
3. Missing Distributed Cache (Redis): State is kept in single-process memory, breaking clustering and autoscaling.
4. Missing Private S3 Architecture: PHI documents use public URLs instead of short-lived presigned GET URLs.
5. Missing Network Isolation: EC2 and database are not isolated in private subnets behind a NAT gateway.

Ø<ß¯ TOP 10 MANDATORY PRE-PRODUCTION FIXES (PRIORITIZED BY IMPACT)

1	**Remove OTP from API Responses** Eliminate data.otp across all authentication endpoints; integrate real SMS gateway.
2	**Lock Down POST /api/users** Require Admin authentication; strip client-supplied role parameters in public routes.
3	**Remediate Mass BOLA / IDOR** Remove optionalJWT from /api/patients; enforce record ownership (citizen_user_id === user.id).
4	**Enforce Database TLS (rejectUnauthorized: true)** Supply Root CA bundle to PostgreSQL pool; enable certificate chain verification.
5	**Protect Notification Broadcasting** Require verifyJWT and authorizeRoles("ADMIN") on /api/mob/notifications/broadcast.
6	**Enforce 100% Private S3 Storage** Block public bucket access; stream medical documents exclusively via presigned GET URLs.
7	**Tighten CORS Whitelist** Remove *.vercel.app regex; allow only exact company-owned production FQDNs.
8	**Disable Cleartext Traffic on Android** Set android:usesCleartextTraffic="false" in AndroidManifest.xml; mandate HTTPS.
9	**Purge Plaintext Credentials from Logs** Remove console.log/logger.info calls printing OTPs, passwords, and user tokens.
10	**Sanitize HTML in Report Printing (XSS)** Sanitize hospital names in Reports.jsx using DOMPurify before document.write.

Ø=ÝÓþ RECOMMENDED 5-PHASE IMPLEMENTATION ROADMAP

Phase 1: Critical (Days 1–3)	Strip leaked OTPs; lock POST /api/users; enforce BOLA ownership on /patients; secure broadcast alerts.
Phase 2: High (Days 4–7)	Enforce DB TLS; disable Android cleartext; tighten CORS; privatize S3; sanitize HTML in Reports.jsx.
Phase 3: Medium (Days 8–14)	Deploy Redis cluster; integrate rate-limit-redis; implement file-type magic byte check; lower body limits to 2MB.
Phase 4: Hardening (Days 15–21)	Migrate to HttpOnly cookies; implement refresh token rotation; expand AuditLogModel to all PHI access.
Phase 5: Cloud Infra (Days 22–30)	Deploy AWS WAF; terminate TLS on ALB; use IAM instance profiles; add Sentry and CloudWatch alarms.

FORMAL SECURITY AUDIT CONCLUSION & COMPLIANCE DISCLAIMER

Your current codebase is approximately 23% security-ready based on the inspected controls. This static source-code audit evaluated code and configuration integrity. Prior to processing live citizen protected health information (PHI) in production, dynamic application security testing (DAST), third-party penetration testing, and cloud infrastructure compliance audits are strictly required.